

Module: - 02

Transport - Layer Services

→ Transport - layer :

The transport layer is core component of the network stack, responsible for logical communication b/w applications on different hosts. It allows applications to send messages to each other as if they were directly connected, regardless of the underlying physical network. The transport layer is implemented in end systems (like your computer or a server), not in network routers.

• Key Functions

- 1) Segmentation : on the sending side, the transport layer breaks down large application messages into smaller transport-layer segments.
- 2) Encapsulation : It adds a transport-layer header to each segment. This header contains information needed for delivery & processing.
- 3) Pass off to Network layer : The segment is then passed to the network layer, where it's encapsulated in a network-layer packet (datagram) & sent across the network.
- 4) De-encapsulation & Delivery : on the receiving side, the network layer extracts the segment from the datagram & delivers it to the transport layer. The transport layer then processes the segment & makes the data available to the receiving application.

• The Internet offers two primary transport-layer protocols:

1) TCP (Transmission Control Protocol): A reliable, connection-oriented protocol that ensures data is delivered in order & without errors. It's used for applications that require high reliability, like web browsing, email, & file transfers.

2) UDP (User Datagram Protocol): A faster, connectionless protocol that doesn't guarantee delivery or order. It's used for applications that prioritize speed over reliability, such as streaming video, online gaming, & voice calls.

→ Relationship b/w Network Layer & Transport Layer

Aspect	Network Layer	Transport Layer
Main role	Provides logical communication b/w hosts	Provides logical communication b/w processes (application)
Where it runs	In routers & end systems	Only in end systems [not in routers]
Unit of data	Packet [IP datagram]	Segment [TCP/UDP]
Addressing	Uses IP addresses (host-level)	Uses port numbers [process-level]
Services offered	Best-effort delivery [IP routing]	Reliability, multiplexing/demultiplexing, error control, flow control, sometimes security [TCP, UDP, etc.]
Guarantees	Cannot guarantee delay, bandwidth, or reliability	Can sometimes provide reliability [TCP], ordering, error recovery even if the network is unreliable
Example	IP (IPv4, IPv6)	TCP, UDP

→ Functions of Transport Layer

1) Reliability :

This ensures that data sent from one application process arrives at the destination process without loss, corruption, or duplications, & in the correct order. The TCP provides this service by using techniques like sequence numbers, acknowledgements, & retransmissions. The ~~User Datagram~~ UDP, in contrast, is an unreliable protocol and does not guarantee delivery.

2) Multiplexing / Demultiplexing :

i) Multiplexing is the process of collecting data from various application processes at the sending host, encapsulating it with a header, & passing the resulting segments to the network layer.

ii) Demultiplexing is the process of delivering the data in a transport-layer segment to the correct application process at the receiving host. This is achieved using port numbers in the transport header, which act as a unique identifier for each application process.

3) Error control & Congestion control :

i) Error control involves detecting & correcting errors that occur during data transmission. Both TCP & UDP use checksums in their headers to detect if a segment's data has been corrupted. However, TCP goes further by retransmitting corrupted or lost segments to ensure reliable delivery.

ii) Congestion control is mechanism used by TCP to prevent the network from becoming overloaded with excessive traffic.

4) Flow Control:

This service, provided by TCP, prevents a sender from overwhelming a receiver with too much data too quickly. The receiving host informs the sending host of its available buffer space, & the sender adjusts its transmission rate accordingly to avoid data overflow & loss.

5) Sometimes Security [TCP, UDP, etc]

While not a primary function of TCP or UDP themselves, security can be provided at the transport layer by protocols like Transport Layer Security [TLS], which is often used in conjunction with TCP.

→ UDP

- Type: unreliable, connectionless transport protocol
- Services provided:
 - i) Multiplexing / Demultiplexing [Process-to-process delivery]
 - ii) Error checking [checksum]
- What it does not provide:
 - Reliability [data may be lost or corrupted]
 - ordered delivery
 - Congestion control.
- Traffic behavior: Applications using UDP can send at any rate [unregulated]
- Use cases: Real-time applications [VoIP, video streaming, DNS, online gaming]

→ TCP

- Type: Reliable, connection-oriented transport protocol.
- Services Provided:
 - i) Reliable data transfer [ensures correct, in-order delivery]
 - ii) Flow control [prevents overwhelming the receiver].

iii) Congestion control [Prevents network overload,
fair sharing of bandwidth]

iv) ~~Error~~ Error checking [checksum]

v) Multiplexing / Demultiplexing

- Traffic behavior: Sending rate is ~~regulated~~ regulated to avoid congestion

- Use cases: web browsing [HTTP/HTTPS], email [SMTP, IMAP, POP3], file transfers [FTP]

⇒ UDP Segment Structure

Source Port #	Dest. Port #
length	checksum
Application Data [message]	

The UDP Segment Structure is defined in RFC 768 & has a simple header with four fields, each two bytes long:

- i) Source Port : This field identifies the port number of the Sending Process.
- ii) Destination Port : This field identifies the port number of the receiving Process. These two fields enable the demultiplexing function, allowing the receiving host to deliver the data to the correct application.
- iii) length : This field indicates the total length of the UDP segment in bytes, including both the header & the data.
- iv) checksum : This field is used to detect any errors that may have been introduced into the segment during transmission.

- Ports → identify Processes
- length → segment size
- checksum → error detection
- Data field → application-specific content

→ UDP checksum calculation

• Checksum at Sender Side

- i) Break the message:
Divide the original message into k blocks,
each of n bits [e.g., 16 bits]
- ii) Sum the blocks:
Add all k blocks together using binary addition
- iii) Handle overflow [carry]:
• If there's a carry (overflow beyond n bits), wrap it around & add it back
- iv) Take 1's complement:
• Invert all bits of the final sum → this becomes the checksum

• Checksum at Receiver Side

- i) collect all data blocks + checksum
• Receiver takes the k data blocks sent by the sender
Plus the checksum
- ii) Add them all together:
• Perform binary addition of all the blocks & the checksum
• If there's a carry, wrap it around & add back
[just like sender side]
- iii) check the result:
• If the result is all 1's (111...), it means no error
→ ACCEPT the data.
• If the result is not all 1's error is detected →
REJECT the data.

→ Performance of checksum

i) Good at detecting odd bit errors

If 1, 3, 5, ... bits get corrupted, checksum will almost always detect it

ii) Detects most even bit ~~even~~ errors

If 2 or 4 bits flip randomly, many times checksum can still detect it

But not Perfect [limitations]

- If some bits are damaged in one block & opposite bits are damaged in another block, the sum doesn't change

- In that case, the receiver will think data is correct → error goes undetected.

- What happens if error is found?

i) UDP does not correct errors

ii) Damaged segment may be:

- Discarded by some implementations

- Delivered with a warning to the application by others.

→ Principles of Reliable Data Transfer:

Reliable data transfer is a fundamental problem in networking, occurring at multiple layers [transport, link, application]. The goal is to create a reliable channel for data transfer over an unreliable underlying channel. This means that data must be delivered to the receiver without being corrupted or lost, & in the correct order.

• Framework & Terminology

The reliable data transfer protocol is often referred to as rdt (reliable data transfer Protocol).

- i) `rdt_send()`: The function called by the upper layer at the sending host to initiate the transfer of data
- ii) `rdt_rcv()`: The function called when a Packet arrives at the receiving host from the unreliable channel below
- iii) `deliver_data()`: The function used by the receiving side of the rdt protocol to pass received data up to its application layer
- iv) `udt_send()`: The function used by both the sender & receiver to send packets across the underlying unreliable channel.

This section focuses on unidirectional data transfers [data moving from sender to receiver] but acknowledges that control packets [e.g., acknowledgements] need to flow in both directions. The discussion will incrementally build a reliable Protocol, starting with a simple, reliable channel & moving to more more complex ~~see~~ scenarios where the underlying channel can corrupt or lose packets. ~~we~~ we'll initially assume that packets are delivered in order, even if some are lost. The general principles learned here apply directly to protocols like TCP

→ Building a Reliable Data Transfer Protocol:

- In ~~the~~ CN's, data is transmitted over communication channels that may introduce errors, loss or duplication
- To ensure that the data received is exactly the same as what was sent, reliable data transfer (RDT) Protocols are used

- The first step in building a reliable data transfer protocol is to assume the simplest case: a perfectly reliable channel. The resulting protocol, called rdt1.0 is straightforward & serves as a foundational model

rdt1.0 Sender:

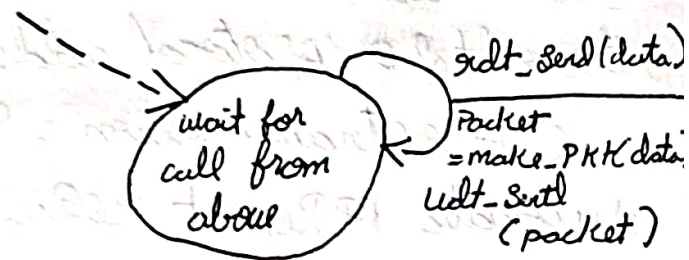
The sender's logic is defined by a single-state finite-state Machine [FSM]

- State: Waiting for a call from the upper layer.

- Event: `rdt_send(data)`. This event occurs when the application layer passes data to the sender

- Actions: The sender performs two actions:

- `make_pkt(data)`: creates a packet containing the data
- `udt_send(packet)`: sends the packet over the channel to the receiver



Since the channel is perfectly reliable, the sender doesn't need to wait for any confirmation. It simply sends the data & is ready for the next piece of data from the application

rdt1.0 Receiver:

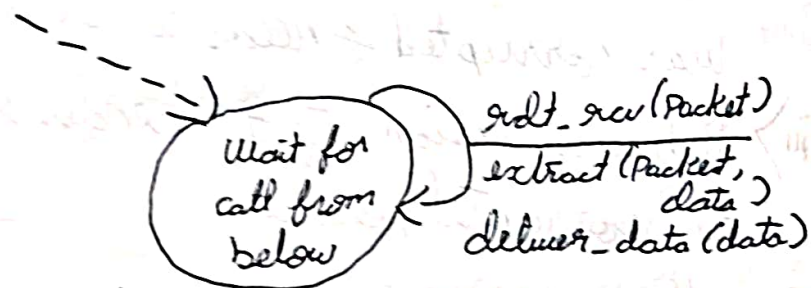
The receiver's logic is also a single-state FSM.

- State: Waiting for a packet from the channel.

- Event: `rdt_recv(packet)`. This event occurs when a packet arrives from the channel.

- Actions: The receiver performs two actions:

- `extract(packet, data)`: Extracts the data from the received packet
- `deliver_data(data)`: Delivers the extracted data to the application layer



Since the channel is perfect, the receiver doesn't need worry about lost, corrupted, or out-of-order packets. It simply processes each

as it arrives

• Why rdt 1.0 is simple

- i) No feedback (No ACK/NAK)
- ii) No difference b/w data & Packet
- iii) NO flow control is required

• rdt 2.0:

To handle a channel where bits in a packet can be corrupted, the rdt 2.0 protocol adds error detection, receiver feedback, & ~~re~~ retransmission. This type of Protocol is called Automatic Repeat Request [ARQ].

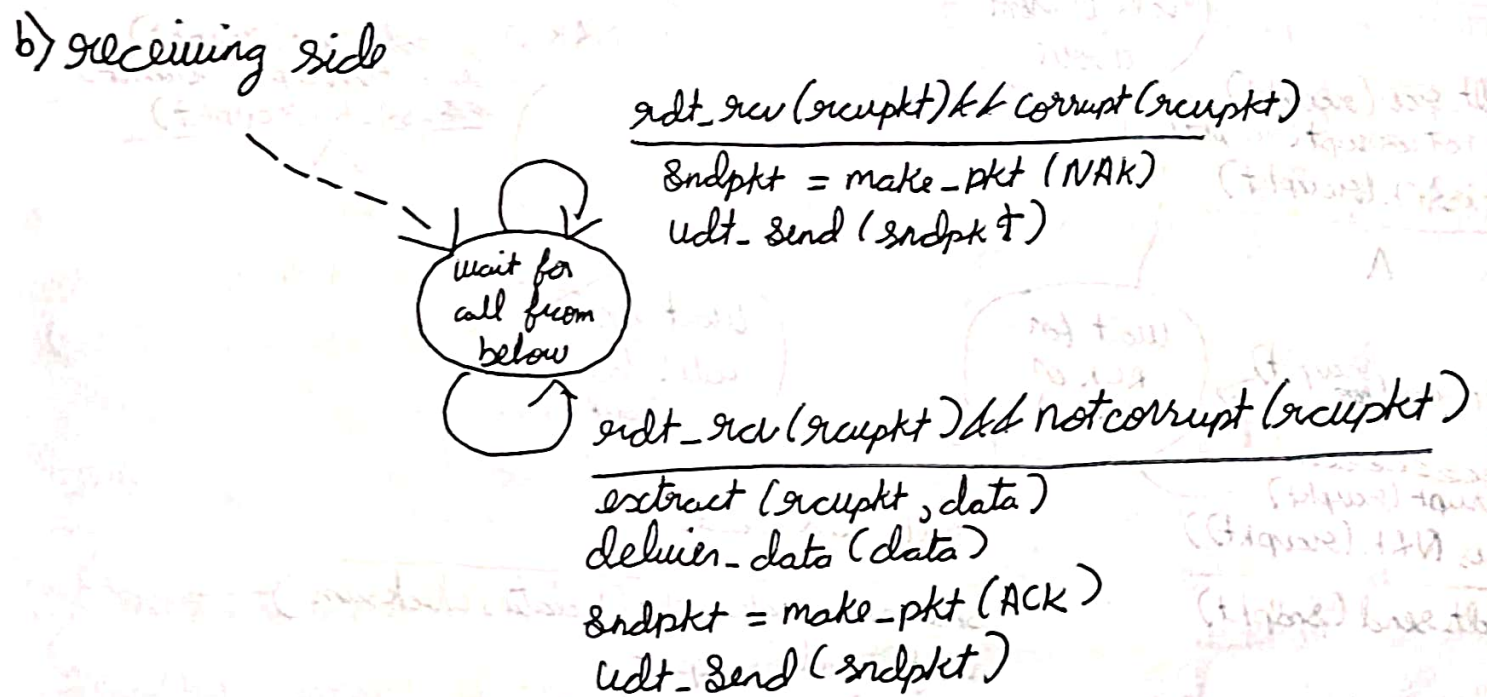
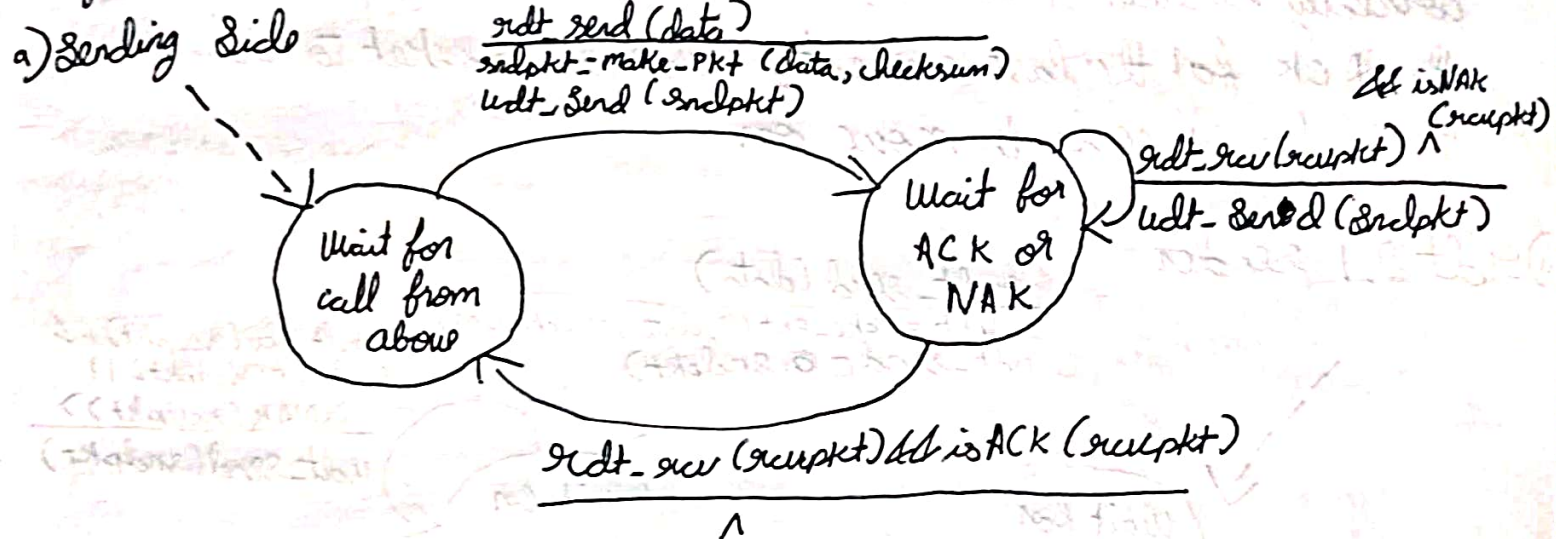
rdt 2.0 Protocol:

- i) Error Detection: The sender includes a checksum in the Packet header. The receiver computes the checksum upon arrival & detects if the packet is corrupted.
- ii) Receiver Feedback: The receiver sends control messages back to the sender:
 - i) ACK (Positive Acknowledgment): Indicates that a packet was received correctly.
 - ii) NAK (Negative Acknowledgment): Indicates that a packet was corrupted & needs to be retransmitted.
- iii) Retransmission: If the sender receives a NAK, it retransmits the corrupted packet.

The flaw & its Fixes:

The fatal flaw in rdt 2.0 is that it doesn't account for a corrupted ACK or NAK packet. If a sender receives a garbled ACK/NAK, it doesn't know what the receiver's state is.

To fix this, we must handle the possibility of corrupted feedback
 - A simple solⁿ is to have the sender retransmit the data packet upon receiving a garbled ACK/NAK. However, this introduces the packets of duplicate packets, since the receiver won't know if a retransmitted packet is new data or a retransmission



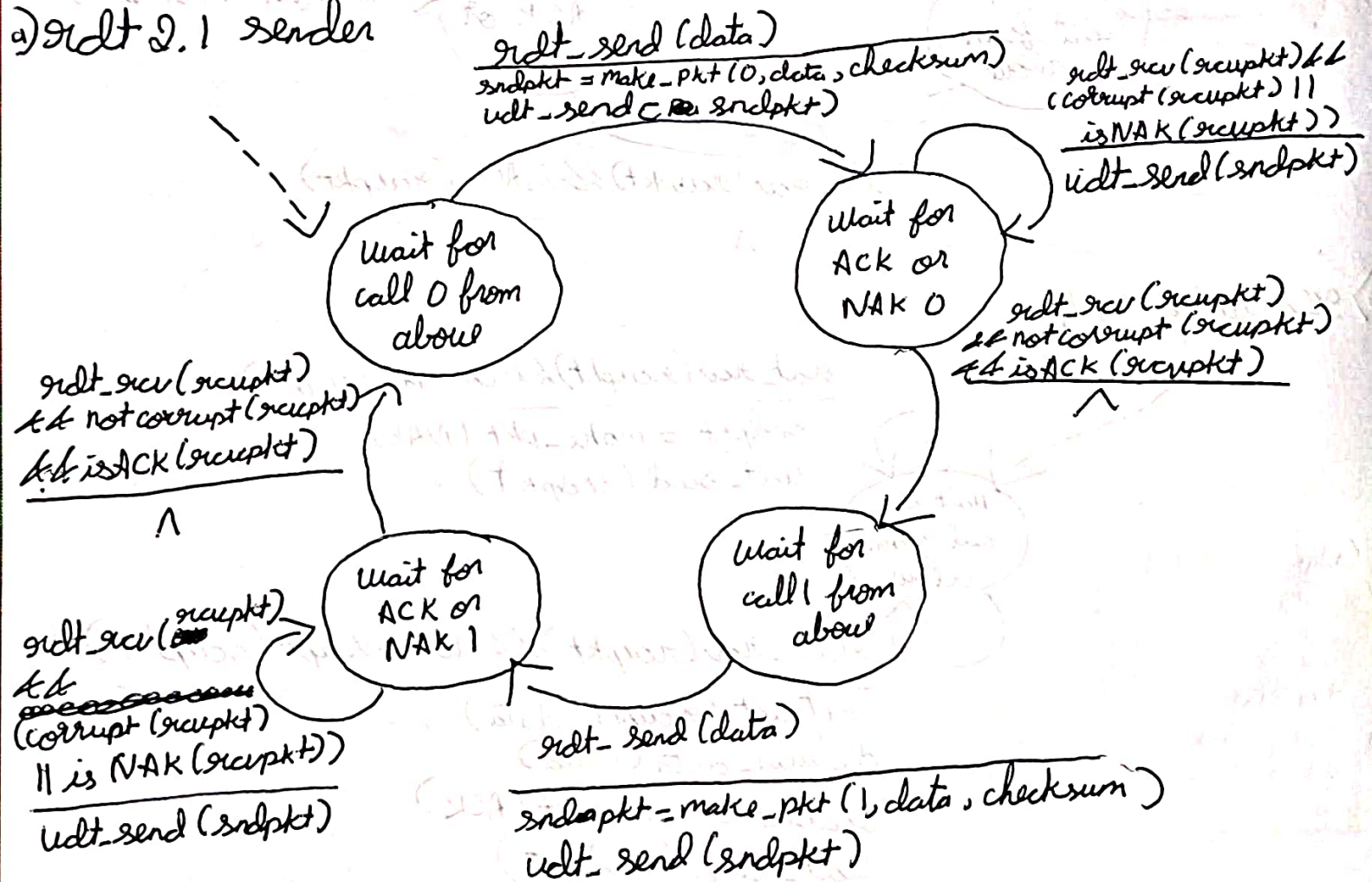
rdt 2.1: Adding Sequence Numbers:

The solution to the duplicate packet problem is to add a sequence number to the data packets with a simple stop-and-wait Protocol, a single bit (0 or 1) is enough.

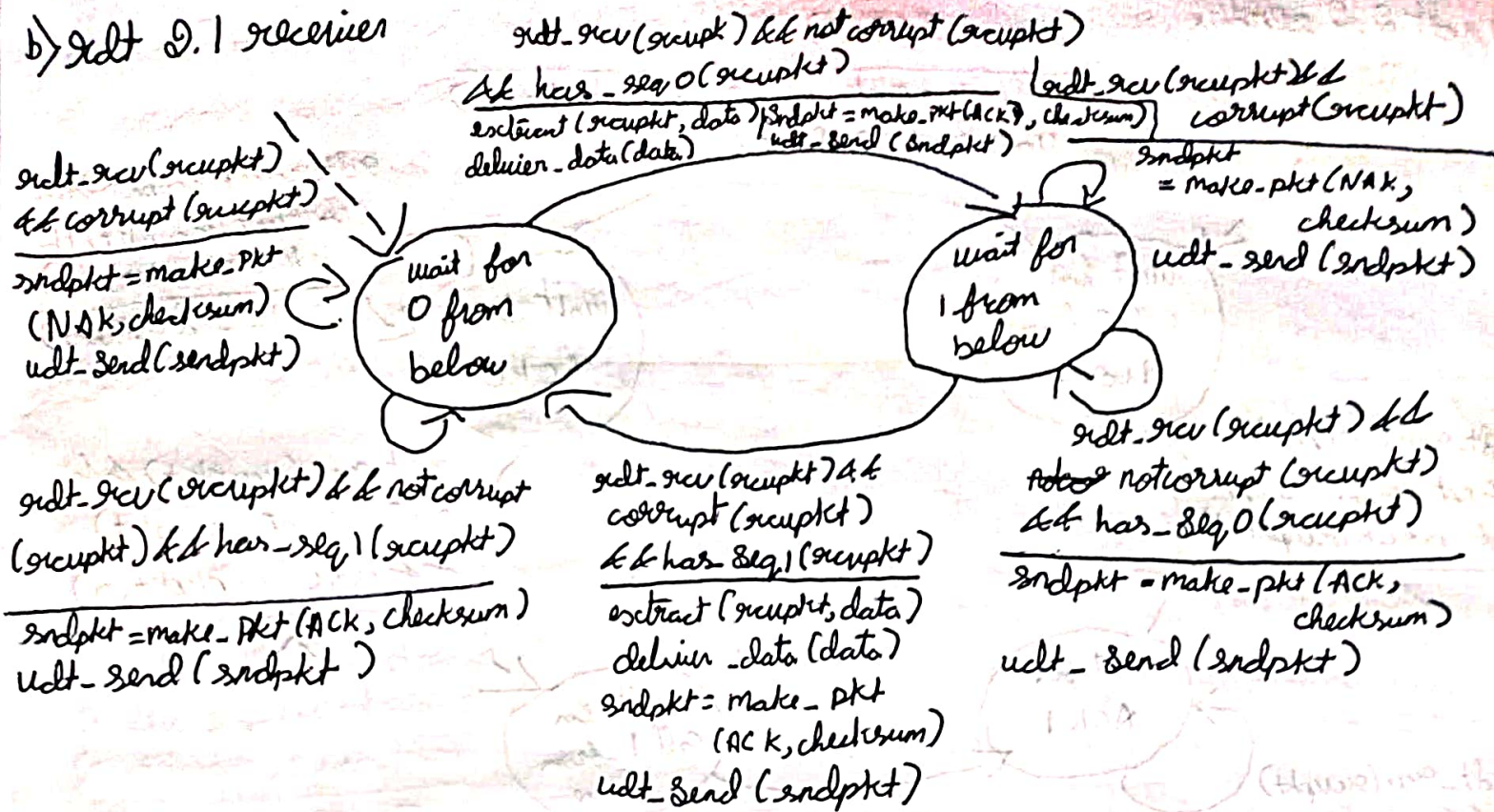
i) Sender: Attaches a sequence number (0 or 1) to each new Packet. It retransmits the same Packet (with the same sequence number) if an ACK/NAK is lost or corrupted

- Receiver: checks the sequence number of the arriving packet
 - i) If the sequence number is the one it's expecting, it accepts the data & sends an ACK for the sequence number
 - ii) If the sequence number is a duplicate (the same as the last correctly received packet), it discards the data but re-sends the ACK for the last correctly received packet to inform the sender it needs to move on

a) rdt 2.1 sender



b) rdt 2.1 receiver



• rdt 2.2: A-NAK-Free Protocol

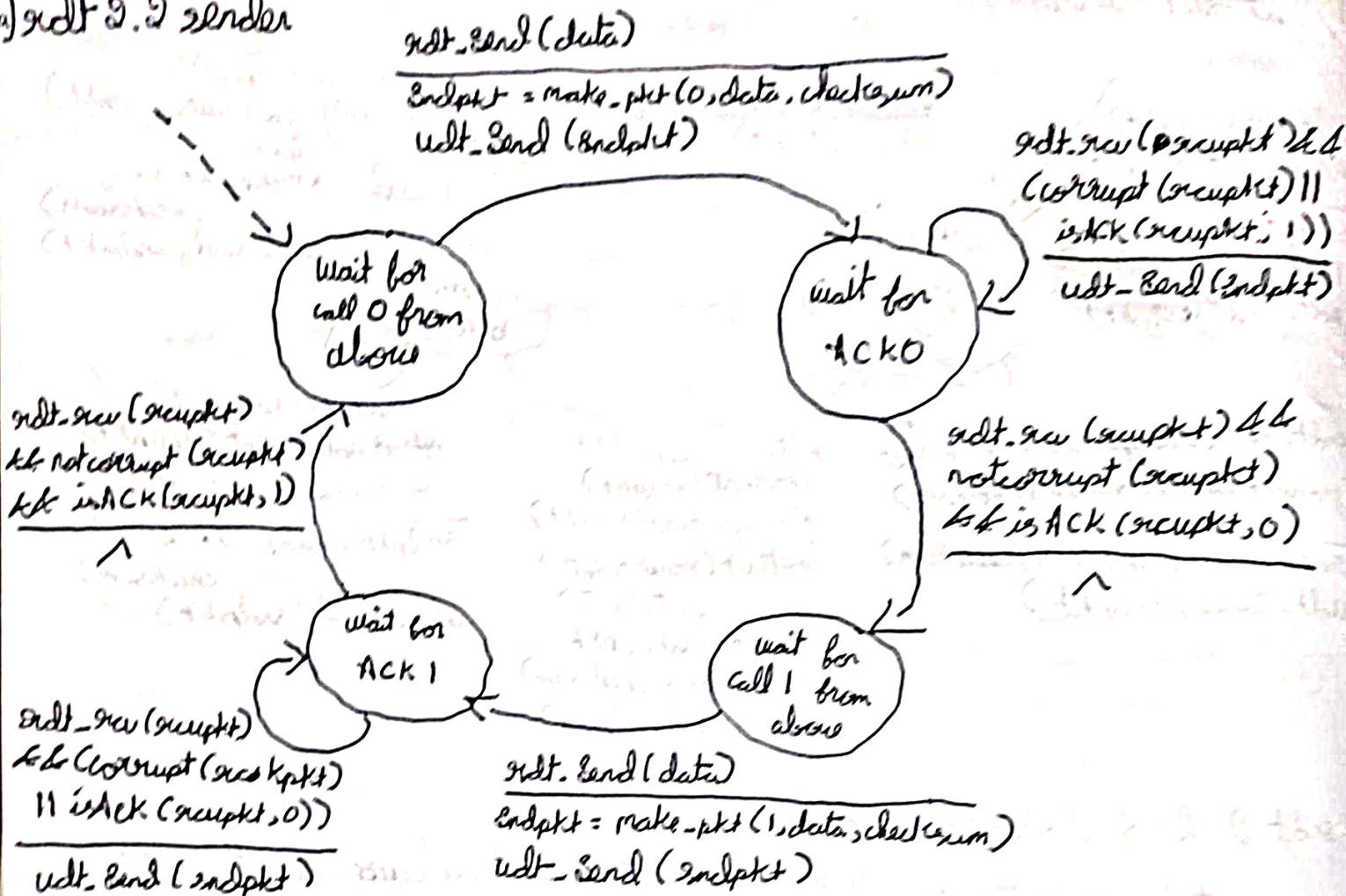
rdt 2.2 is a refined version of rdt 2.1 that achieves the same functionality without using NAKs. Instead of sending a NAK for a corrupted packet, the receiver sends a duplicate ACK for the last correctly received packet.

- i) when the sender receives a duplicate ACK, it understands that the receiver did not correctly receive the next packet & retransmits it
- ii) for this to work, the ACK messages must now include the sequence number of the packet they are acknowledging

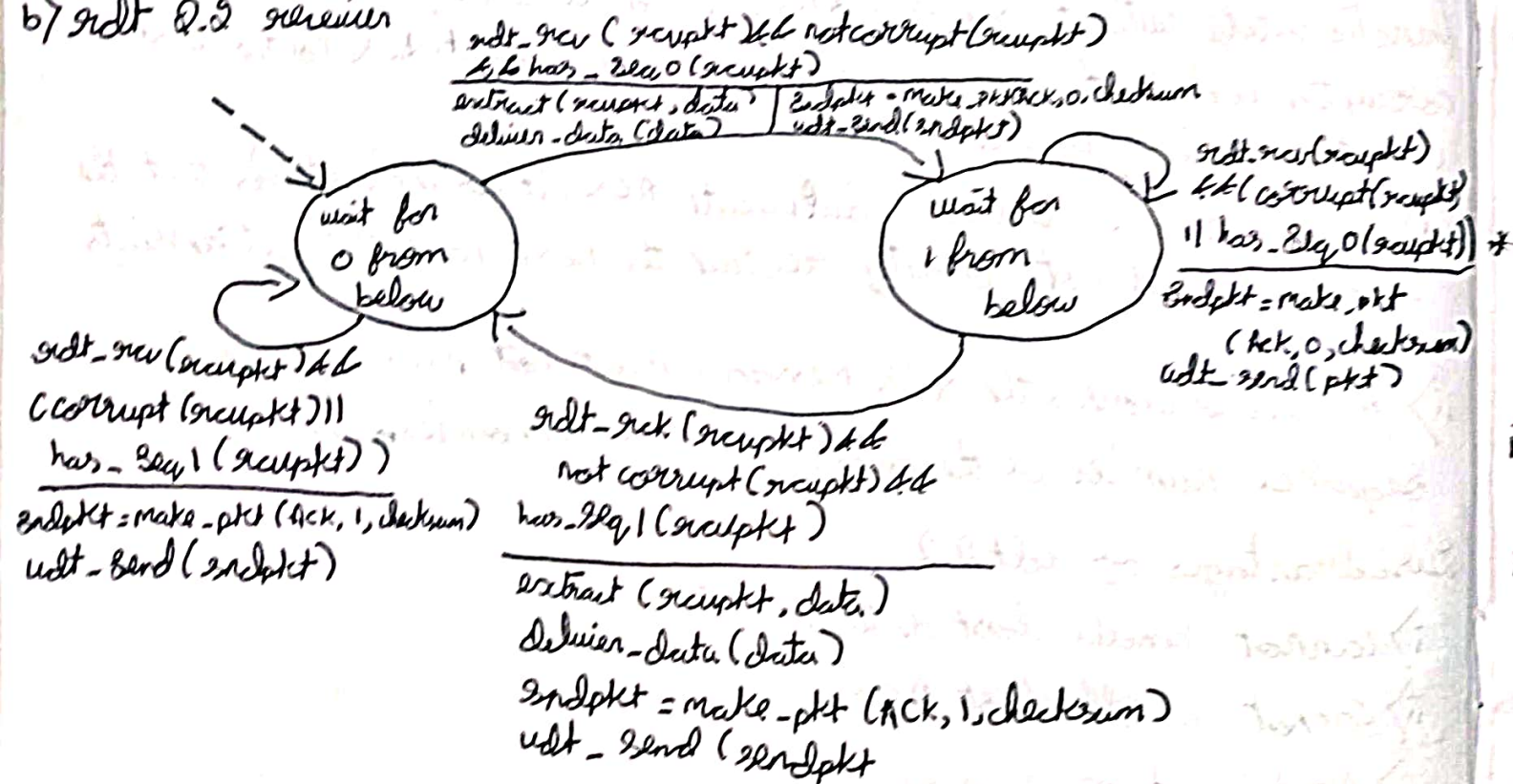
Disadvantages of rdt 2.2

- i) cannot handle lost data packets
- ii) cannot handle lost ACKs
- iii) No timeout mechanism
- iv) Deadlock possibility

a) rdt 2.2 sender



b) rdt 2.2 receiver



• When the underlying channel is lossy (meaning it can lose packets), a new mechanism is needed to detect & recover from these losses. The rdt 3.0 protocol adds a countdown timer to handle packet loss.

* Detecting Packet Loss:

The sender is responsible for detecting loss. Since it can't know for sure if a packet or its corresponding ACK was lost, it relies on a time-out mechanism.

- i) When the sender transmits a packet, it starts a timer.
- ii) If the timer expires before an ACK is received, the sender assumes the packet has been lost & retransmits it.

* The Paradox of Retransmission:

The sender doesn't distinguish between a lost data packet, a lost ACK, or a delayed packet. The action is the same in all cases: ~~retransmits~~ retransmits, the receiver's existing sequence number logic [from rdt 2.2] handles the ~~data~~ duplicate packet by simply discarding it & re-sending the ACK for the last correctly received packet.

* rdt 3.0 sender FSM:

- The sender's logic now includes the management of a timer.
- i) When the sender transitions to the "waits for ACK" state, it starts a timer.
 - ii) If a valid ACK is received, the sender stops the timer & moves on to the next packet.
 - iii) If the timer expires [a timeout event], the sender retransmits the packet & restarts the timer.

Because this protocol uses alternating sequence numbers (0 & 1) for packets, it is also known as the alternating-bit protocol. With the addition of timers, checksums, sequence numbers, & acknowledgements, the rdt 3.0 protocol provides a functional & reliable data transfer service over lossy channels.

rdt 3.0 Sender

